Improving RAG with Reranking, Chunking, and Conversational Memory

Naga Sai Haritha Gottumukkala
Homework 10 · DATA 266, San José State University
November 9, 2026

***Abstract*—In this report I tried three simple ways to help a RAG system find better text. I took the first 100 papers from the arXiv summarization dataset. I cut them into small pieces in two different ways and saved the pieces in ChromaDB. Then I compared normal search with search plus a reranker. I also compared a chatbot that forgets the last question with one that remembers it. Reranking raised Precision@5 from 0.65 to 0.70. Recursive chunking did a little better than fixed-size chunking. Memory made the biggest difference, and Precision@5 went from 0.40 to 0.80. My test was small, so these results are only a first look.**

***Index Terms*—retrieval-augmented generation, reranking, chunking, conversational memory, ChromaDB, Precision@K, MRR.**

I. INTRODUCTION

A RAG system answers a question in two steps. First it looks up pieces of text. Then a language model writes an answer from those pieces [6]. If the lookup finds the wrong pieces, the answer will be bad, even if the model is very good. In this homework I only looked at the lookup step. I tried three changes and asked one question about each:

1) Reranking. If a second model checks the first 20 search results again, do the top 5 get better?

2) Chunking. Does the way I cut the documents change what is found?

3) Memory. If the system remembers the last question, can it handle a follow-up like "How do they help?"

Fig. 1 shows the whole pipeline. The orange boxes are the parts I tested.

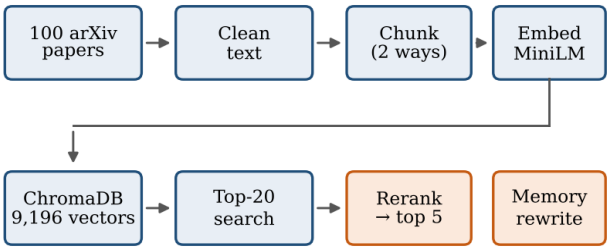


Fig. 1. Pipeline used in this homework.

II. DATA AND PREPROCESSING

I used the ccdv/arxiv-summarization dataset from Hugging Face. I took the document version and the first 100 papers from the train split [1]. All 100 papers were different, and none were empty. The papers are long. The average is about 36,100 characters, or 6,500 words. The shortest has 3,400 characters and the longest has 193,100. For cleaning, I only removed extra spaces. I kept the rest of the text as it was.

III. METHODOLOGY

A. Two Chunking Strategies

Both strategies use chunks of up to 1000 characters, with 200 characters of overlap. The only difference is where the cuts are made.

Strategy 1, fixed-size: cut after every 1000 characters, even in the middle of a sentence.

Strategy 2, recursive: LangChain's RecursiveCharacterTextSplitter [2] tries to cut at the end of a paragraph first. If that does not work, it cuts at the end of a sentence, and then at a word.

Table I shows the results. Recursive chunking made more chunks, and they were shorter. Also, 36 of them had fewer than 200 characters, and the smallest had only 9. Pieces this small say almost nothing useful.

TABLE I. CHUNK STATISTICS (CHARACTERS)

Strategy	Chunks	Mean	Median	Min	<200
Fixed-size	4,536	991.6	1000	212	0
Recursive	4,660	873.9	916	9	36

B. Embedding and Indexing

I turned each chunk into a list of 384 numbers (a vector) using the all-MiniLM-L6-v2 model [3]. I

saved all 9,196 vectors (4,536 + 4,660) in ChromaDB [4]. It finds similar chunks using cosine distance.

C. Retrieval and Reranking

I used two questions. Both are about the first article in the dataset:

Q1: *"What are the advantages of additive models compared with fully nonparametric models?"*

Q2: *"How is quantile regression used with regularized kernel-based methods?"*

The normal search just takes the 5 most similar chunks. For reranking, I take the top 20 chunks first. A cross-encoder model (ms-marco-MiniLM-L-6-v2) [5] reads the question together with each chunk and gives it a new score. I then keep the best 5. I tested both chunking strategies, so there are 4 setups in total.

D. Conversational Memory

My memory code keeps the user's questions and saves the last topic. If a new question uses a word like "they" or "it", the code swaps in the saved topic. My test chat had two turns:

Turn 1: *"What are the advantages of additive models?"* (topic saved: additive models)

Turn 2: *"How do they help in high-dimensional problems?"*

After the rewrite, Turn 2 becomes *"How do additive models help in high-dimensional problems?"* I searched with Turn 2 twice, once as it was and once rewritten. I used fixed-size chunks for this test.

E. Evaluation

I checked every retrieved chunk by hand. I gave it 1 if it would help answer the question and 0 if not. This gave 80 judgments for retrieval (38 relevant) and 17 for memory (5 relevant). I used three measures:

- **Precision@5:** how many of the top 5 chunks are useful.
- **Pooled Recall@5:** useful chunks in the top 5, divided by all useful chunks I found in the pool. This is not recall over the whole database, because I could not label all of it.
- **MRR:** the average of 1/rank for the first useful chunk. A first-place hit scores 1.

IV. RESULTS

A. A Closer Look: Q1, Fixed-Size

Table II lists the real chunks for Q1. Reranking changed only one result. Chunk 7 of document 0 moved from rank 7 up to rank 5. It pushed out chunk 10, which was not useful. This one swap raised Precision@5 from 0.6 to 0.8.

TABLE II. Q1 TOP 5, FIXED-SIZE CHUNKS

#	Without reranking	With reranking
1	d0·c0 ✓	d0·c0 ✓
2	d0·c23 ✓	d0·c23 ✓
3	d0·c3 ✗	d0·c3 ✗
4	d0·c16 ✓	d0·c16 ✓
5	d0·c10 ✗	d0·c7 ✓

d0 = document 0, c = chunk number, ✓ relevant, ✗ not relevant.

B. All Four Setups

TABLE III. RETRIEVAL SETUPS (AVERAGE OF Q1 AND Q2)

Setup	P@5	Recall@5	MRR
Fixed, no rerank	0.60	0.339	0.750
Fixed, reranked	0.70	0.411	1.000
Recursive, no rerank	0.70	0.369	1.000
Recursive, reranked	0.70	0.386	1.000

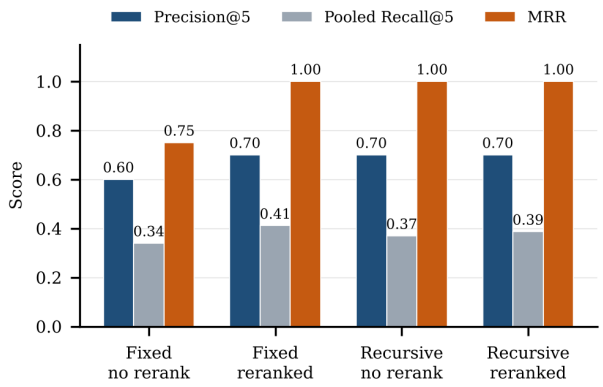


Fig. 2. Precision@5, pooled Recall@5 and MRR for all four setups.

Fig. 3 shows the top 5 for every run. Green cells are useful chunks. You can quickly see where reranking helped and where it did not.

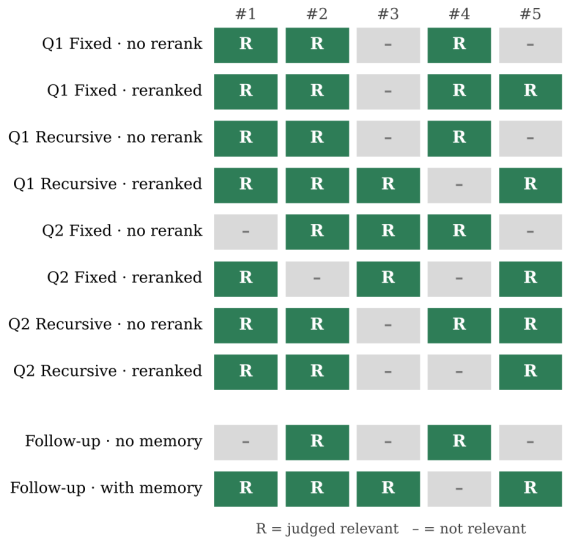


Fig. 3. Relevance of the top 5 chunks for each run, by rank.

C. Without vs. With Reranking

Reranking improved every measure when I averaged both chunking strategies (Table IV). The biggest gain was in MRR, which went from 0.875 to 1.000. This means a useful chunk was always in first place.

TABLE IV. WITHOUT VS. WITH RERANKING

Approach	P@5	Recall@5	MRR
Without reranking	0.650	0.354	0.875
With reranking	0.700	0.399	1.000

But reranking did not help every time (Table V). In three of the four cases it helped or made no difference. In one case it made things worse. For Q2 with recursive chunks, precision dropped from 0.8 to 0.6. The reranker added chunk 0, which was not useful, and removed chunk 24, which was useful.

TABLE V. PER-QUERY RESULTS (P@5 / RECIPROCAL RANK)

Strategy	Query	No rerank	Reranked
Fixed	Q1	0.6 / 1.0	0.8 / 1.0
Fixed	Q2	0.6 / 0.5	0.6 / 1.0
Recursive	Q1	0.6 / 1.0	0.8 / 1.0
Recursive	Q2	0.8 / 1.0	0.6 / 1.0

D. Fixed-Size vs. Recursive Chunking

Recursive chunking scored a little higher on precision (0.70 vs. 0.65) and on MRR (1.00 vs. 0.875). Recall was about the same (Table VI). I only used two questions, so I see this as a small edge and not a clear win.

TABLE VI. FIXED-SIZE VS. RECURSIVE CHUNKING

Strategy	P@5	Recall@5	MRR
Fixed-size	0.650	0.375	0.875
Recursive	0.700	0.378	1.000

E. Without vs. With Memory

Without memory, the follow-up "How do they help?" found unrelated chunks. The top result came from document 84, which is a paper about quadratic differentials. Ranks 3 and 5 also came from other papers. With the topic filled in, four of the five results came from the right paper. The first three were all useful (Table VII, Fig. 4).

TABLE VII. FOLLOW-UP QUERY, FIXED-SIZE CHUNKS

#	Without memory	With memory
1	d84·c4 ✗	d0·c23 ✓
2	d0·c31 ✓	d0·c0 ✓
3	d93·c10 ✗	d0·c16 ✓
4	d0·c23 ✓	d0·c10 ✗
5	d83·c22 ✗	d0·c31 ✓

TABLE VIII. MEMORY COMPARISON

Approach	P@5	Recall@5	RR
Without memory	0.40	0.40	0.50
With memory	0.80	0.80	1.00

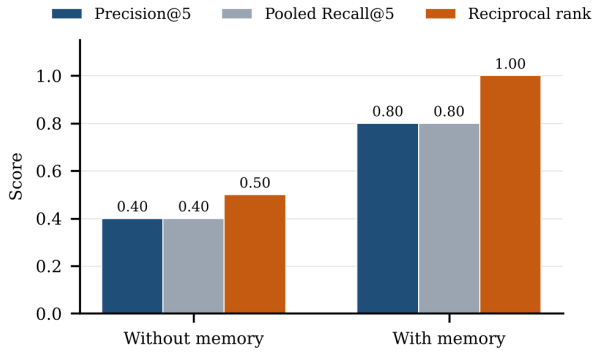


Fig. 4. Effect of conversational memory on the follow-up query.

V. DISCUSSION

Memory gave the biggest improvement by far (+0.40 precision). This makes sense, because a question like "How do they help?" means almost nothing on its own. Reranking came next. It helps because the first search looks at the question and the chunk separately, while the cross-encoder reads them together. The downside is speed, since 20

extra chunks must be scored for every question. Chunking made the smallest difference.

I also noticed something else. For Q2, the top 20 chunks came from only 1 or 2 papers. For Q1 they came from 6 papers. When most chunks come from one paper, the reranker has little to choose from. This may be why the gains were small.

VI. FAILURE ANALYSIS

TABLE IX. PROBLEMS FOUND AND POSSIBLE FIXES

Problem	Evidence	Fix
Rerank hurt	Recursive Q2: P@5 0.8 → 0.6	Mix both scores
Vague follow-up	P@5 and recall only 0.40	LLM query rewrite
Tiny chunks	36 under 200 chars; min 9	Merge short ones
Few queries	2 queries, 1 paper	More topics
Small pool	Reranker saw 20 only	Hybrid search

VII. LIMITATIONS

I used only two search questions and one memory chat, so a single chunk can change the scores a lot. I was the only person who judged relevance, and both questions came from one article. Recall is only measured inside the top-20 pool. My memory method is a simple rule, and it would fail on harder follow-up questions. Please treat the differences as hints and not as proof.

VIII. CONCLUSION

Memory gave the clearest gain, with precision going from 0.40 to 0.80. Reranking raised precision from 0.65 to 0.70 and put a useful chunk in first place every time, though it made one result worse. Recursive chunking was slightly better than fixed-size chunking, but it also made a few tiny, useless chunks. Next, I would test more questions from more papers, ask a second person to judge relevance, merge the short chunks, and try a better way to rewrite follow-up questions.

REFERENCES

[1] A. Cohan et al., “A discourse-aware attention model for abstractive summarization of long documents,” in Proc. NAACL-HLT, 2018 (dataset: ccdv/arxiv-summarization, Hugging Face).

[2] LangChain, “RecursiveCharacterTextSplitter,” langchain-text-splitters documentation.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019.

[4] Chroma, “ChromaDB: the AI-native open-source embedding database,” documentation.

[5] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.

[6] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.